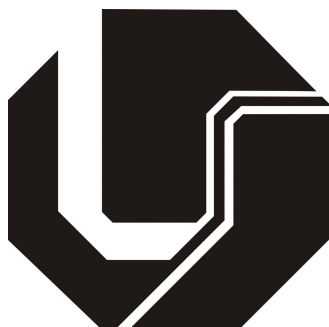


Universidade Federal de Uberlândia
Sistemas de Informação
Sistemas Distribuídos



Festival de Rock Distribuído

Eduardo Silveira Alvim Santos - 31911BSI004

Monte Carmelo

2025

Sumário

1	Estado atual do Projeto	3
2	Justificativa para a escolha entre processos e threads	3
3	Comunicação via Sockets	3
4	Execução do Código	4

1 Estado atual do Projeto

Nesta fase, estabeleci a primeira interação funcional entre dois dos principais agentes do nosso microambiente distribuído: o Fã e o Palco.

Fiz um pequeno exemplo conectando essas duas entidades, utilizando comunicação via sockets TCP. Ele simula o comportamento de um fã que se conecta a um palco com o objetivo de receber atualizações sobre o show e enviar mensagens em retorno.

Essa implementação está diretamente ligada à Etapa 1, na qual foi definida a estrutura do sistema com múltiplos palcos, fãs e quiosques. Neste momento, o foco esteve em validar a base da comunicação entre os agentes, aspecto fundamental para garantir o funcionamento distribuído e dinâmico que caracteriza o nosso microambiente.

2 Justificativa para a escolha entre processos e threads

As escolhas entre utilizar processos ou threads foram orientadas pelas particularidades e funções atribuídas a cada tipo de entidade:

Entidade	Implementação	Justificativa
Fã	Thread	Espera-se um número elevado de fãs, e seu comportamento é mais leve. Utilizar threads facilita o compartilhamento de dados e tem menor custo.
Palco	Processo	Cada palco é uma entidade independente com estado próprio. A separação por processos garante isolamento, tolerância a falhas e escalabilidade.

Tabela 1: Justificativa para decisões sobre processos e threads

Essa distinção está ligada à arquitetura definida na Etapa 1, em que os fãs atuam de forma autônoma e descentralizada, enquanto os palcos funcionam como serviços independentes e autogerenciados dentro do sistema.

3 Comunicação via Sockets

Os componentes utilizados foram Fã, que será nosso Cliente TCP, e Palco, que será nosso servidor TCP multithread. O TCP foi escolhido porque a troca de mensagens entre o fã e o palco exige confiabilidade e ordem. Estamos lidando com comunicações estruturadas e importantes — como saudações, atualizações de status e respostas — e, por isso, é fundamental garantir que as mensagens cheguem corretamente e na sequência esperada. Por exemplo: o fã precisa receber a confirmação de que o show começou no momento certo. Nesse contexto, o TCP se mostra o protocolo ideal. As mensagens são trocadas como strings simples codificadas em UTF-8.

Exemplos de mensagem do palco:

- *Bem-vindo ao Palco! O show começa em breve.*

- *O show começou, ENTRE NO MOSH!*

Exemplos de mensagem do fã:

- *To na área, derrubou é pênalti!*
- *Obrigado pelo show!*

Essa dinâmica de mensagens simula, de forma simples, a comunicação inicial entre os agentes do micro-mundo, refletindo tanto a funcionalidade quanto o espírito do ambiente distribuído.

4 Execução do Código

Antes de começar, certifique-se de ter Python 3 instalado em sua máquina e um editor de código ou acesso a um terminal (Linux ou Windows). Para executar o código basta seguir o passo a passo:

1. Abra dois terminais
2. Vá até a pasta em que os arquivos estão armazenados em ambos os terminais
3. No primeiro terminal, inicie o servidor do palco com o comando: `python3 palco_tcp.py`
4. No segundo terminal, execute o cliente fã com: `python3 fa_tcp.py`